buzzFest

Developer Technical Documentation

Architecture, Design, Interfaces, Setup and Operations (As Built)

Document control

Product	buzzFest (concept name: Apna Bakery), a marketplace for home bakers in Bangalore
Live site	https://buzz-fest.vercel.app/
Repository	https://github.com/rnijampure/buzzFest (branch: main, last commit 14 Oct 2026)
Document type	Developer technical documentation (HLD and LLD, as built)
Version / status	1.0 Draft. Beta proof of concept
Companion document	Product Documentation (PRD style): page behaviour, copy and acceptance criteria
Verified against	Repository source, Prisma migrations, Neon table exports, live site (18 Oct 2026)

Revision history

Version	Date	Change
0.1	18 Oct 2026	Foundation (scope, architecture, data model)
1.0	18 Oct 2026	Complete document: design, interfaces, components, setup, deployment, limitations, future scope

Contents

1. Introduction **3**
1.1 Purpose 3
1.2 Scope 3
1.3 Project status 3
1.4 Conventions 3
2. System Overview **3**
2.1 What the system does 3
2.2 Constraints and assumptions 3
2.3 Technology stack (from `package.json`) 3
3. Architecture **4**
3.1 Logical architecture 4
3.2 Repository layout 5
3.3 Page-to-code map 5
3.4 Server and client boundaries 5
3.5 Rendering and caching 6
3.6 Database connection (`lib/prisma.ts`) 6
4. Data Design **6**
4.1 Entity relationships 6
4.2 Tables 7
4.3 Design decisions 8
4.4 Schema drift (three sources disagree) 9
4.5 Seed data (Neon export) 9
5. Detailed Design **10**
5.1 Catalogue query and filter pipeline 10
5.2 Search component 11
5.3 Cart state (`store/store.tsx`) 11
5.4 Pricing rules 12
5.5 Order creation (`app/actions/checkout.ts`) 12
5.6 Product detail 13
5.7 Home composition 13
5.8 Images 13
6. Interface Specifications **13**
6.1 Public routes 13
6.2 Server Action 14
6.3 Internal module interfaces 14
7. Component Library **14**
7.1 Structure 14
7.2 Primitives (`components/ui/`) 14
7.3 Building blocks 14
7.4 Design tokens (`app/globals.css`) 14
7.5 Testing and documentation of components 15
8. Security **16**
9. Performance **16**
10. Accessibility **16**

11. Environment, Setup and Commands 16
11.1 Prerequisites 16
11.2 Environment variables 17
11.3 First-time setup 17
create .env with the variables above 17
11.4 Scripts 17
12. Deployment and Operations 17
12.1 Topology 17
12.2 Release procedure (manual) 17
12.3 Troubleshooting 18
12.4 Monitoring and logging 18
13. Known Limitations Register 18
14. Future Scope (Not Planned for Build) 19
Appendix A. Draft HLD/LLD PDF: Reconciliation 19
Appendix B. Glossary 20
Appendix C. Screenshots to Add 20
Appendix D. Open Items for the Owner 21

1. Introduction

1.1 Purpose

This document describes how buzzFest is built and how to run, test, deploy and extend it. It is the reference a developer uses to understand the system. **What each page should do, and the on-screen copy, is described in the Product Documentation**; this document links behaviour to code.

1.2 Scope

In scope: the customer storefront (Next.js `app/`), the database model, the order-creation server action, shared components and hooks, tests, and deployment.

Out of scope: the operations/baker dashboard (not started), payment gateway, authentication, notifications.

1.3 Project status

buzzFest is a free-tier **beta proof of concept**. The customer journey ends at the cart. It is documented as it exists. Future work is collected in §14 and is not planned for build.

1.4 Conventions

Label	Meaning
[VERIFIED]	Confirmed in source code and, where possible, on the live site
[CONFIRM]	Could not be verified from the material available; needs the owner to check
KL-nn	Entry in the Known Limitations register (§13)

Paths are relative to the repository root. `@/` is the alias for the root folder.

2. System Overview

2.1 What the system does

Customers browse products from four independent bakers, filter and search the catalogue, view product detail, and build a cart. A server action can write an order and baker tasks to the database.

2.2 Constraints and assumptions

#	Constraint / assumption
C1	Free tiers only: Vercel free plan, Neon free tier, Cloudinary free plan
C2	Neon free tier limits connections; the app uses a pool of 2 and tolerates cold starts
C3	No user accounts: orders are placed with a placeholder customer
C4	Catalogue is small (4 bakers, 28 products), so all filtering is done in database queries without pagination
C5	All prices are in Indian rupees, whole numbers

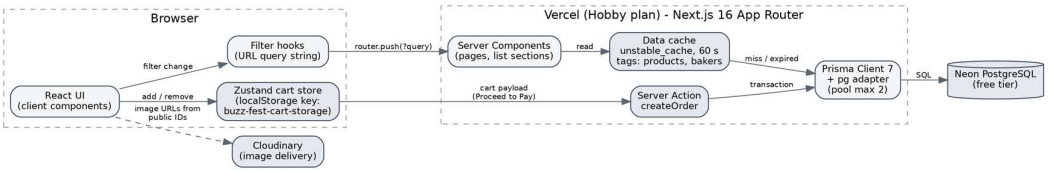
2.3 Technology stack (from `package.json`)

Layer	Technology	Version
Framework	Next.js (App Router)	^16.2.4
UI library	React	19.2.4

Layer	Technology	Version
Language	TypeScript	5.x [CONFIRM]
Styling	Tailwind CSS, `tw-animate-css`	^4
Component kit	shadcn/ui (style `radix-nova`) on Radix UI	radix-ui ^1.4.3
Icons	lucide-react	^1.7.0
Carousel	embla-carousel-react, embla-carousel-autoplay	^8.6.0
Client state	Zustand with `persist` middleware	^5.0.12
ORM	Prisma Client with `@prisma/adapter-pg` and pg	^7.8.0
Database	PostgreSQL on Neon	n/a
Images	Cloudinary via `next-cloudinary` (`CldImage`)	^6.17.5
Unit and component tests	Vitest, happy-dom	^4.1.5
Component workbench	Storybook (`@storybook/nextjs-vite`), a11y addon, Chromatic addon	^10.3.6
Browser tests	Playwright (through Vitest)	^1.59.1
Linting	ESLint 9 with `eslint-config-next`	16.2.1
Hosting	Vercel (free plan)	n/a

3. Architecture

3.1 Logical architecture



System architecture

Tier	Responsibility	Implementation
Presentation	Render pages and interactive widgets	Server Components for data pages; Client Components for filters, cart, carousel, image zoom
Application	Business rules and queries	Query modules `getAllProducts.ts`, `GetAllBakers.ts`; Server Action `checkout.ts`
Data access	Database connection and queries	`lib/prisma.ts` (Prisma Client with a pg pool)
Persistence	Relational data	Neon PostgreSQL
Static assets	Images	Cloudinary (public IDs stored in the database)

There are **no REST or GraphQL API routes**. Reads happen inside Server Components. The only write is the `createOrder` Server Action.

3.2 Repository layout

```
/
├── app/                          Next.js App Router (routes and page-level components)
│   ├── layout.tsx                Root layout: Header, children, CartSummary, CartSheet, Footer
│   ├── page.tsx                  Home
│   ├── about_us/                 About Us
│   ├── bakers/                   Our Bakers (force-dynamic)
│   ├── cart/                     Cart (client component)
│   ├── products/                 Listing, query module, [slug]/ detail
│   ├── actions/checkout.ts       Server Action: createOrder
│   ├── feature/Bakers/           Cached baker query
│   ├── component/layout/         Header, footer, landing sections, cart sheet and icon
│   ├── component/UI/             Cards, carousel, badges, filters, slider, star, buttons
│   └── globals.css               Design tokens and Tailwind theme
├── components/ui/                shadcn/ui primitives (button, card, sheet, sidebar, ...)
├── hooks/                        useUpdateFilters, useDebounce, useCart, use-mobile
├── lib/                          prisma.ts, types, utils, star-rating, badge-filter-colors
├── store/store.tsx               Zustand cart store
├── types/product.ts              Product type and display helpers
├── prisma/                       schema.prisma, migrations/, seed.ts, seedProducts.ts
├── public/                       Static images, logo
├── .storybook/                   Storybook configuration
├── next.config.ts                serverExternalPackages: @prisma/client, pg
├── prisma.config.ts              Prisma datasource: DIRECT_URL, falling back to DATABASE_URL
├── vitest.config.ts              Test projects: unit, cards, storybook
└── apps/, .nx/                   Build and cache artefacts (see KL-23)
```

3.3 Page-to-code map

Route	Entry file	Data source	Rendering
/	app/page.tsx → LandingPage.tsx	BakersList (GetAllBakers({}), first 4); DoughnutsList (category doughnut); CupcakesList (category cakes, custom); carousel uses constants	Server, Suspense per section
/products	app/products/page.tsx	GetAllProducts(filters)	Server per request (reads searchParams)
/products/[slug]	app/products/[slug]/page.tsx → ProductClient.tsx	prisma.product.findUnique with baker	Server fetch, client form
/bakers	app/bakers/page.tsx	BakersList	force-dynamic
/about_us	app/about_us/page.tsx	None (static copy)	Server
/cart	app/cart/page.tsx	Zustand store	Client

Layout and chrome (app/layout.tsx): sticky header with navigation and cart badge, floating cart quick-view tab, cart side sheet, footer placeholder. Fonts: Montserrat via next/font/google.

3.4 Server and client boundaries

Concern	Runs on	Files
Catalogue and baker queries	Server	products/getAllProducts.ts, feature/Bakers/GetAllBakers.ts
Filter state	Client, reflected in the URL	hooks/useUpdateFilters.ts, productsFilter.tsx, NavigationHelpers.tsx, Badge.tsx, slider.tsx, components/ui/search.tsx
Cart	Client, saved in localStorage	store/store.tsx, cart/page.tsx, cartSideSheet.tsx, shoppingCart.tsx

Concern	Runs on	Files
Hero carousel	Client, desktop only, after mount	`BakeryFeaturedCarousel.tsx`
Order creation	Server Action	`actions/checkout.ts`

3.5 Rendering and caching

Area	Mechanism	Detail
Product list	`unstable_cache`	Key `["products-list", JSON(filters)]`, `revalidate: 60`, tag `products`. Each filter combination caches separately
Baker list	`unstable_cache`	Key `["bakers", "bakers-{minRating}-{location}"]`, `revalidate: 60`, tag `bakers`
`/bakers` route	`export const dynamic = "force-dynamic"`	Rendered on every request
Product detail	None	Direct Prisma call on each request
Home	Streaming	Each section inside `<Suspense>` with a skeleton fallback
Mobile hero	Single eager, high-priority image	Fast Largest Contentful Paint; interactive carousel mounts on desktop only
Images	Cloudinary `f_auto`, `q_auto`	Format and quality chosen automatically

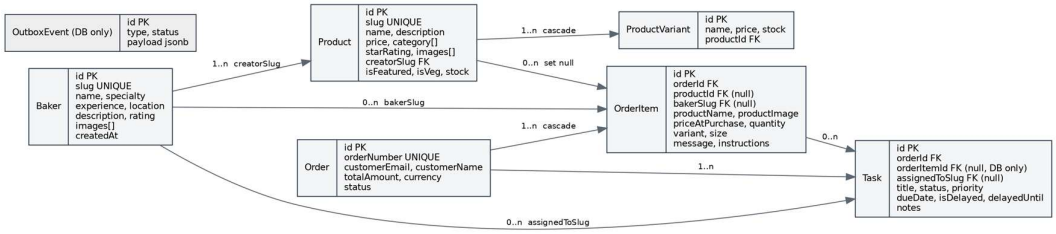
No code calls `revalidateTag`, so catalogue changes appear after at most 60 seconds. **[CONFIRM]** whether `unstable_cache` is still the recommended API in Next.js 16 (the repository's `AGENTS.md` warns that this Next.js version differs from older releases).

3.6 Database connection (`lib/prisma.ts`)

- Uses `PrismaClient` with the `PrismaPg` adapter and a `pg.Pool` limited to **2 connections** with a 15-second connection timeout, to stay inside the Neon free tier.
- The client is cached on `globalThis` outside production to avoid duplicate clients during hot reload.
- Logging: queries, warnings and errors in development; errors only otherwise.
- `prisma.config.ts` uses `DIRECT_URL` (direct connection) for Prisma CLI commands, falling back to `DATABASE_URL`.

4. Data Design

4.1 Entity relationships



Entity relationship diagram

Baker links use **slugs**, not ids: `Product.creatorSlug`, `OrderItem.bakerSlug` and `Task.assignedToSlug` all reference `Baker.slug`. Slugs must stay unique and stable.

4.2 Tables

Baker

Column	Type	Notes
id	text, PK	cuid
name, specialty, experience, location, description	text	`experience` is free text, for example “Baker since 2012”
slug	text, unique	Join key
images	text[]	Cloudinary public IDs
rating	double precision	Default 5.0
createdAt	timestamp	

Product

Column	Type	Notes
id	text, PK	cuid
name, description	text	
slug	text, unique	Used in URL `/products/{slug}`
price	decimal (10,2) per migrations	Base price in INR
category	text[]	Free-form tags for type, diet, occasion and marketing labels
starRating	double precision	Default 0
images	text[]	Cloudinary public IDs; the first is primary
creatorSlug	text, FK to Baker.slug	`ON DELETE RESTRICT`
isFeatured, isVeg	boolean	`isVeg` default true
stock	integer	Product-level stock, default 0
createdAt, updatedAt	timestamp	

ProductVariant

Column	Type	Notes
id	text, PK	
name	text	For example `500g`, `1kg`, `Box of 6`, `Pack of 12`
price	decimal (10,2)	Price for this variant
stock	integer	
productId	text, FK	`ON DELETE CASCADE`

The table is populated (59 rows, covering 20 of 28 products) but **no code reads it** (KL-04).

Order

Column	Type	Notes
id	text, PK	
orderNumber	integer, unique	Customer-facing number
customerEmail	text	Required
customerName	text, nullable	

Column	Type	Notes
totalAmount	decimal (10,2)	Items plus delivery fee
currency	text	Default `INR`
status	enum `OrderStatus`	See §4.4
createdAt, updatedAt	timestamp	

OrderItem (immutable purchase snapshot)

Column	Type	Notes
id	text, PK	
orderId	text, FK	`ON DELETE CASCADE`
productId	text, nullable FK	`ON DELETE SET NULL`: history survives product deletion
bakerSlug	text, nullable	FK to Baker.slug in the live database
productName, productImage	text	Snapshot
priceAtPurchase	decimal (10,2)	Snapshot
quantity	integer	Default 1
variant, size, message, instructions	text, nullable	Cart options

Task (baker work queue)

Column	Type	Notes
id, title	text	
orderId	text, FK	`ON DELETE RESTRICT`
assignedToSlug	text, nullable FK	Baker.slug
status	enum	`QUEUED`, `IN_PROGRESS`, `COMPLETED`
priority	enum	`LOW`, `MEDIUM`, `HIGH`, `URGENT`
dueDate	timestamp	
isDelayed, delayedUntil	boolean, timestamp	
notes	text	
orderItemId, notesToSelf, customerName, customerContact		In the live database only

OutboxEvent: exists in the live database only (`type`, `status`, `payload jsonb`). Not used by any code.

4.3 Design decisions

Decision	Reason
`OrderItem` copies name, image and price	Past orders must not change when the catalogue changes
`OrderItem.productId` nullable with `SET NULL`	Deleting a product must not delete order history
One `Task` per baker per order	A multi-baker order splits into separate work queues
Tags in a `text[]` instead of a category table	Fast to extend; costs consistency (KL-07)
Cart in the browser	No database writes while browsing
Baker joins by slug	Readable URLs and fewer joins; slugs must never change

4.4 Schema drift (three sources disagree)

Item	schema.prisma and migrations	Live database export (schema.txt)
Money columns	Decimal(10,2)	integer **[CONFIRM]** (exported values are whole numbers either way)
OrderStatus	PENDING, PAID, PREPARING, OUT_FOR_DELIVERY, COMPLETED, CANCELLED	PENDING, COMPLETED, CANCELLED
Task.notes, Task.delayedUntil	Required	Nullable
Extra Task columns	Absent	orderItemId, notesToSelf, customerName, customerContact
OrderItem.bakerSlug	Plain optional text	Foreign key with index
OutboxEvent table, OutboxStatus enum	Absent	Present

The shared schema.prisma and prisma/migrations agree with each other (three migrations, 6 Apr, 8 Apr and 25 Apr 2026). The exported database does not match them, so it was changed outside Prisma Migrate or from a different branch. **Treat as a release risk (KL-18)**: run prisma migrate diff against the live database before any schema work.

4.5 Seed data (Neon export)

Table	Rows	Notes
Baker	4	Sandhya Mishra (slug sandhya-mandi), Priya Sharma, Meenakshi Iyer, Ananya Das
Product	28	Priya 8, Meenakshi 7, Sandhya 7, Ananya 6; 8 featured; 2 out of stock
ProductVariant	59	Names: 500g/1kg/2kg, Pack of 3/6/12, Box of 4/6/12, Single Loaf, Pack of 2
Order / OrderItem	4 / 8	All PENDING, all customer@example.com

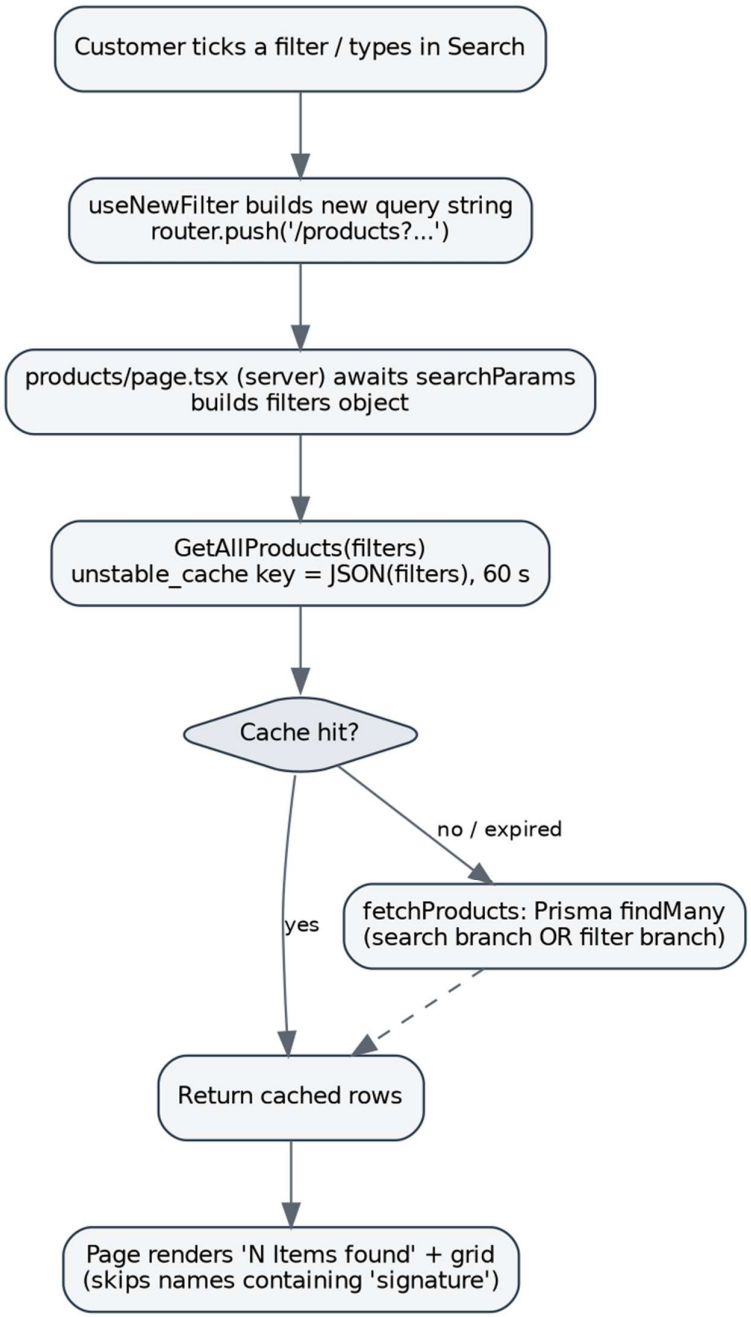
Every exported order total equals the sum of priceAtPurchase × quantity plus a flat ₹70 delivery fee, which matches the cart page rule.

Seeding: prisma/seed.ts and prisma/seedProducts.ts (run with npx tsx). Neither script deletes existing rows. **[CONFIRM]** idempotency before running against a populated database.

Data quality issues (KL-19): trailing tab characters in the Cocomelon Delight slug and the “Browny The Bear” name; mixed-case slug Lavender-lemonade-drop; shared image IDs between two products; baker display name derived from slug (“Sandhya Mandi”) differs from the stored name (“Sandhya Mishra”); tag casing inconsistent (Limited, Best Seller, Eggless versus lowercase elsewhere).

5. Detailed Design

5.1 Catalogue query and filter pipeline



Filter and query flow

URL contract (all parameters optional):

Parameter	Format	Example	Server behaviour
`category`	Comma-separated tags	`category=cake,vegan`	Lower-cased and normalised, then `category hasSome` the values. If the list contains `vegan`, `hasEvery` is used instead

Parameter	Format	Example	Server behaviour
`creatorSlug`	Comma-separated baker slugs	`creatorSlug=priya-sharma,ananya-das`	Spaces to hyphens, lower-cased, `creatorSlug in [...]`
`price`	`min,max`	`price=100,500`	`price gte min` and `price lte max` on the **base price**
`search`	Free text	`search=chocolate`	Replaces all other filters. Case-insensitive `contains` on name, description and baker slug, or exact tag match

Normalisation (`getAllProducts.ts`): lower-case, trim, whitespace or underscore to a single hyphen, collapse repeated hyphens.

Query selection logic 1. `filters` empty: `prisma.product.findMany()`. 2. `search` present: search branch with an `OR` over name, description, creatorSlug and category. 3. Otherwise: filter branch combining creator, category, price and `isFeatured` conditions with `AND`.

Page-level step (`products/page.tsx`): if the URL has `search`, only `search` is passed; otherwise `category`, `creatorSlug` and `price` are parsed. The page then **skips any product whose name contains "signature"**, and renders "N Items found" using the unfiltered array length.

Filter UI state is derived directly from the URL (`useSearchParams`), with no local copy, so a link reproduces a filtered view. The hook `useNewFilter` exposes `updateFilters`, `toggleUpdateFilter` and `clearAllFilters`. `toggleUpdateFilter` also stores the lower-cased parameters in `localStorage` under `oldParams` (not read anywhere in the shared code).

Price slider: local range state (₹100 to ₹2,000, step ₹50), debounced by 500 ms before updating the URL, with URL-to-state synchronisation for reset.

Behaviour that differs from what a user may expect

Case	Result	Ref
Tag matching	Exact and **case-sensitive** against stored tags. `limited` does not match `Limited`; `best-seller` does not match `best seller` or `Best Seller`	KL-07
Multi-word search	The search term is normalised with spaces turned into hyphens, so "chocolate cake" searches for `chocolate-cake` and finds nothing in names containing spaces	KL-26
`Featured` and `Exclude Out Of Stock`	These sidebar items add `category=featured` and `category=exclude-out-of-stock`; no product has those tags	KL-05
Star rating filter	Not connected	KL-06
Price	Filters the base price; variant prices are ignored	KL-04

5.2 Search component

`components/ui/search.tsx` sends an update on each key release and again 400 ms after typing stops (two code paths). Searching clears all other parameters. The input is not prefilled from the URL. See KL-26.

5.3 Cart state (`store/store.tsx`)

Type `CartItem`: `id`, `name`, `price`, `originalPrice?`, `image`, `quantity`, `specialInstructions?`, `variant?`, `size?`, `message?`.

Member	Behaviour
`cartItems`	Array of items
`open`	Side sheet open flag (not persisted)

Member	Behaviour
`addToCart(item)`	If an item with the same `id` exists, adds the quantity to it (options are ignored). Otherwise inserts a new item with defaults: `originalPrice = price + 50`, `variant = "Default"`, `message = "No Message given"`, `specialInstructions = "No special requests"`
`removeFromCart(id)`	Removes the line
`incrementQty(id)`, `decrementQty(id)`	Changes quantity; a quantity of 0 removes the line
`clearCart()`	Empties the cart
`setIsCartOpen(bool)`	Opens or closes the side sheet

Persistence: Zustand `persist` with `localStorage`, key `buzz-fest-cart-storage`, saving only `cartItems`. The cart page waits for mount before rendering to avoid hydration mismatches; `hooks/useCart.ts` provides the same protection but the header badge reads the store directly. The badge shows `cartItems.length` (number of lines, not units).

Add to Cart (`addToCartButton.tsx`) builds an item from `id`, `name`, `image` (first image), `price`, `quantity` (default 1), `message`, `specialInstructions`, `size`. It **does not include `creatorSlug`** (see §5.5 and KL-11) and does not check stock (KL-17).

5.4 Pricing rules

Rule	Cart page	Side sheet
Delivery fee	₹70, free when item total is above ₹1,500	₹40, free when item total is above ₹500
Tax	"Tax Included" label, none added	5% added
"Savings"	Not shown	Uses `originalPrice`, which is always price + ₹50

The two components compute their own totals. They are inconsistent (KL-09). The ₹70 rule matches the exported orders.

5.5 Order creation (`app/actions/checkout.ts`)

Signature: `createOrder(cartData: any): Promise<{ success: true; orderId: string } | { success: false; error: string }>`

Input (as sent by `cart/page.tsx`)

Field	Source
`email`, `name`	Hard-coded: `customer@example.com`, `Customer Name`
`total`	`subtotal + deliveryFee`, computed in the browser
`items[]`	The cart items: `id`, `name`, `image`, `price`, `quantity`, `variant`, `message`, `specialInstructions`, `creatorSlug` (see note)

Processing (single Prisma transaction, `maxWait` 20 s, `timeout` 60 s) 1. Create `Order` with status `PENDING`, `orderNumber` random 100000 to 999999, total as `Decimal`. 2. Create one `OrderItem` per cart line (snapshot of name, image, price, quantity, variant, message, instructions, baker slug). 3. For each **distinct, non-empty** baker slug, create a `Task`: title "New Order #{orderNumber}", priority `MEDIUM`, status `QUEUED`, due in 24 hours, note "Prep for {customerName}". 4. Return `{ success: true, orderId }`.

Errors: any exception is logged (`CRITICAL_CHECKOUT_FAILURE`) and returned as `{ success: false, error: error.message }`. The cart page shows a generic alert.

On success the cart page clears the cart and navigates to `/checkout/success/{orderId}`. **That route does not exist in the repository** (KL-15).

Defects in the current implementation

Defect	Effect	Ref
Server trusts price and total from the browser	A tampered request can set any price	KL-12
`creatorSlug` is not placed in cart items by `AddToCartButton`	`bakerSlug` is null, so no `Task` rows are created	KL-11
Random order numbers with a unique constraint	A collision fails the whole order; no retry	KL-13
No stock check or decrement	Overselling is possible	KL-14
Raw exception text returned to the client	Can leak internal details	KL-12
Fixed customer identity	All orders share one email and name	KL-16

The exported `OrderItem` rows all show `bakerSlug = sandhya-mandi`, even for products made by other bakers, which means those rows were created by an earlier version of the code.

5.6 Product detail

The page loads `prisma.product.findUnique({ where: { slug }, include: { baker: true } })`, converts `price` to a number and renders `ProductClient`. If no product is found it returns the text “Not found”. The image carousel (`productImage.tsx`, Embla) supports thumbnails, hover magnification, and an enlarged view in a modal on click. The form state holds quantity, size, message and instructions and passes them to `AddToCartButton`. Static content in this component is listed in KL-03.

5.7 Home composition

`LandingPage.tsx` renders the carousel, then the three data sections, each in `<Suspense>`. Section titles are rendered with `SectionDivider`. Responsive item counts use Tailwind visibility classes on the first four results (1, 2, 3 or 4 visible by breakpoint).

5.8 Images

Database fields hold Cloudinary **public IDs** (for example `cupcake_nm9qrs`). `CloudinaryImage` wraps `CldImage` from `next-cloudinary`. Some seeded values are local paths (`../assets/images/cupcake/1.jpg` in one order item) or filenames in `public/images`. Alt text is generic in several places (KL-25).

6. Interface Specifications

6.1 Public routes

Method	Route	Parameters	Response
GET	`/`	none	HTML
GET	`/products`	`category`, `creatorSlug`, `price`, `search`	HTML, filtered grid
GET	`/products/{slug}`	slug	HTML, or the text “Not found”
GET	`/bakers`	`minRating`, `location` (read but not applied to the grid)	HTML
GET	`/about_us`	none	HTML
GET	`/cart`	none	HTML (client rendered)
GET	`/checkout/success/{orderId}`	none	Not implemented

6.2 Server Action

`createOrder`: see §5.5. Invoked from the cart page. No HTTP route is exposed by the developer; Next.js generates the endpoint.

6.3 Internal module interfaces

Module	Function	Contract
`products/getAllProducts.ts`	`GetAllProducts(filters?)`	Returns Prisma `Product[]`, cached 60 s
`feature/Bakers/GetAllBakers.ts`	`GetAllBakers({ minRating?, location? })`	Returns `Baker[]` (with `products` when filters are given), cached 60 s
`types/product.ts`	`formatCreatorSlug`, `getProductDisplayPrice`, `getProductPriceNumber`	Display helpers. Price converted with `parseInt`, so decimals are truncated
`lib/star-rating.ts`	`getStarFillPercent`, `getStarGradientId`	Pure functions, unit tested
`lib/badge-filter-colors.ts`	`getFilterBadgeClasses`	Pure function, unit tested
`hooks/useUpdateFilters.ts`	`useNewFilter()`	`updateFilters`, `toggleUpdateFilter`, `clearAllFilters`, `searchParams`
`hooks/useDebounce.ts`	`useDebounce(value, delay)`	Returns the delayed value

7. Component Library

7.1 Structure

Components come in three layers: **primitives** in `components/ui/` (shadcn/ui, Radix based), **building blocks** in `app/component/UI/`, and **layout sections** in `app/component/layout/`.

7.2 Primitives (`components/ui/`)

`aspect-ratio`, `avatar`, `badge`, `button`, `card`, `carousel`, `checkbox`, `CloudinaryImage`, `collapsible`, `field`, `input`, `label`, `navigation-menu`, `search`, `select`, `separator`, `sheet`, `sidebar`, `skeleton`, `slider`, `tooltip`. Button has a custom `brand` variant.

7.3 Building blocks

Component	Purpose	Key inputs
`ProductCard` (`card/ProductCard.tsx`)	Home-page product card with stock and Add to Cart	`product`
`ProductsCard` (`products/ProductsCard.tsx`)	Listing card with price badge, tags and Add to Cart	product fields
`BakerCard`	Baker summary: image, experience badge, name, specialty, rating, location, quote	`baker`
`CardImage`, `StarRating`, `BaseCardSkeleton`	Image with fallback, rating stars, loading skeleton	image id, rating, `hasAction`
`StarIcon` (`star.tsx`)	Fractional star using an SVG gradient	`rating`, `size`
`Badge` (`FilterBadgeList`, `BadgeComponent`)	Tag chips that toggle the `category` filter	`items`, `color` (brown, green, gold)
`PriceSlider`	Debounced price range	none
`NavListItem`, `SidebarMenuSub`	Checkbox rows for sidebar filters	`item`, `type`
`ProductsSidebar`	Collapsible filter sidebar	none

Component	Purpose	Key inputs
`BakeryFeaturedCarousel`, `CarouselClient`, `CarouselShell`	Hero slider (autoplay)	constants `ITEMS`
`ProductImageCarousel`	Product gallery, zoom, modal	`images`, `name`
`AddToCartButton`	Builds a cart item and adds it	product fields plus options
`SectionDivider`	Section heading band	`title`
`CartSheet`, `CartSummaryComponent`, `ShoppingCartComponent`	Side sheet, floating tab, header icon	none (read the store)
`NavMenu` (`header/navigation.tsx`)	Header navigation with Products menu	none

7.4 Design tokens (`app/globals.css`)

Token	Value	Use
`--brand-dark-brown`	`oklch(0.35 0.08 25)`	Header and sidebar background
`--brand-green`	`oklch(0.52 0.12 145)`	Primary action, headings
`--brand-green-light`	`oklch(0.72 0.15 130)`	Tag chips
`--brand-gold`	`oklch(0.82 0.14 72)`	Highlights, selected chips, star fill
`--brand-amber`, `--brand-orange`, `--brand-orange-deep`	oklch values	Accents, active star stroke
`--radius`	3px	Base corner radius
Font	Montserrat (400 to 700) via `--font-sans`; headings 500 and 700	

Layout breakpoints follow Tailwind defaults: product grids use 1, 2, 3 and 4 columns.

7.5 Testing and documentation of components

Tool	Scope	Command
Vitest `unit`	`lib/**/*.test.ts` (star rating, badge colours)	`npx vitest run --project=unit`
Vitest `cards`	`app/component/UI/card/**/*.test.tsx` (happy-dom)	`npm run test:cards`
Vitest `storybook`	Runs every story in Chromium through Playwright	`npm run test-storybook`
Storybook	Stories for ProductCard, BakerCard, CardImage, StarRating, BaseCardSkeleton, SectionDivider, BakersList	`npm run storybook` (port 6006)

There are no tests for pages, filters, the cart store or the server action, and no end-to-end tests (KL-24).

8. Security

Area	Current state	Ref
Secrets	`.env*` is git-ignored. Environment variables are read from the process	Good
Input validation	None on the server action; any typed payload	KL-12
Price integrity	Prices and total come from the browser	KL-12
Authentication, authorisation	None	KL-16
Error disclosure	Raw error text returned to the browser	KL-12
Rate limiting, bot protection	None	KL-27
Security headers	Default Next.js and Vercel only	KL-27

Area	Current state	Ref
Dependencies	Not scanned; no CI	KL-24
Output encoding	React escapes output; no `dangerouslySetInnerHTML` was found in the shared files	Good
Personal data	Orders store email and name (placeholder values today)	n/a

9. Performance

Item	State
Image delivery	Cloudinary automatic format and quality; width-limited URLs
Home page	Streaming sections with skeletons; mobile hero uses one eager image
Data caching	60-second cache on product and baker queries
Database	Pool of 2 connections; Neon cold starts can add seconds on the first request after idle time
Catalogue size	Entire result set is returned and rendered; no pagination (acceptable at 28 products)
Measurement	No Core Web Vitals or performance budget is defined **[CONFIRM]** using Vercel Analytics or Lighthouse

10. Accessibility

Item	State
Semantics and keyboard	Radix primitives provide keyboard support and ARIA for sheet, collapsible, select, checkbox and navigation menu
Alt text	Several images use the text “Image with blur placeholder” (KL-25)
Icon-only buttons	The cart pencil buttons and the side sheet close button have no accessible name (KL-25)
Colour contrast	Not audited. Brown and green brand colours on white need checking; gold text on light backgrounds is at risk
Tooling	Storybook a11y addon is installed
Planned	Font-size controls and a local-language option are intended but not built

11. Environment, Setup and Commands

11.1 Prerequisites

Node.js 20.9 or later (Next.js 16 requirement) **[CONFIRM]**, npm (a `package-lock.json` is present), a Neon PostgreSQL database, a Cloudinary account.

11.2 Environment variables

Variable	Purpose	Used by
`DATABASE_URL`	Runtime database connection (Neon pooled connection string)	`lib/prisma.ts`, seed scripts
`DIRECT_URL`	Direct (non-pooled) connection for Prisma CLI	`prisma.config.ts`
`NEXT_PUBLIC_CLOUDINARY_CLOUD_NAME`	Cloudinary account name required by `next-cloudinary`	`CldImage` **[CONFIRM]** exact name in Vercel settings

Store them in `.env` (never commit) and in the Vercel project settings.

11.3 First-time setup

```
git clone https://github.com/rnijampure/buzzFest.git
cd buzzFest
npm install                 # runs "prisma generate" automatically
# create .env with the variables above
npx prisma migrate deploy   # apply migrations to an empty database
npx prisma db seed          # runs: npx tsx prisma/seed.ts
npm run dev                 # http://localhost:3000
```

11.4 Scripts

Command	Purpose
`npm run dev`	Development server
`npm run build`, `npm start`	Production build and server
`npm run lint`	ESLint
`npm run storybook`, `npm run build-storybook`	Component workbench
`npm run test:cards`	Card component tests
`npx vitest run --project=unit`	Unit tests
`npm run test-storybook`	Story tests in headless Chromium

12. Deployment and Operations

12.1 Topology

Component	Service	Plan
Application	Vercel, project connected to the GitHub repository	Free
Database	Neon PostgreSQL	Free
Images	Cloudinary	Free

12.2 Release procedure (manual)

1. Merge to `main` (Vercel builds on push; preview builds are created per branch **[CONFIRM]**).
2. If the schema changed: run `npx prisma migrate deploy` against the production database before promoting.
3. Open the deployment, check Home, Products (filtered), a product page, and Cart.
4. Roll back by re-promoting the previous Vercel deployment. Database changes are rolled back from Neon's history/branching feature **[CONFIRM]** retention on the free tier.

12.3 Troubleshooting

Symptom	Likely cause	Action
First page load after idle is slow or fails	Neon cold start	Retry; the pool timeout is 15 s
"Too many connections"	Pool exceeded Neon limit	Keep pool at 2; use the pooled DATABASE_URL
A filter returns nothing	Tag case or hyphen mismatch (KL-07)	Compare the URL value with the stored tag
Catalogue edit not visible	60-second data cache	Wait up to 60 s
Product page URL fails	Stray whitespace in slug (KL-19)	Fix the row in the database

Symptom	Likely cause	Action
Prisma CLI cannot connect	DIRECT_URL missing or wrong	Check `.env`
Images missing	Public ID not found in Cloudinary or cloud name not set	Check ID and variable

12.4 Monitoring and logging

No monitoring, error tracking or analytics is configured. Debug `console.log` calls remain in `products/page.tsx` (3), `bakers/page.tsx`, `products/[slug]/page.tsx` and `ProductClient.tsx`, and log request parameters and fetched data to the server console (KL-23). **[CONFIRM]** whether Vercel Analytics is enabled in the project settings.

13. Known Limitations Register

Severity: **H** = could cause wrong data or security exposure; **M** = visible defect or inconsistency; **L** = polish or hygiene.

ID	Area	Limitation	Sev
KL-01	UI	Footer is a placeholder (the word “Footer”)	M
KL-02	SEO	Every page title is “Create Next App”, with the default description	M
KL-03	Product page	Description, “(120 reviews)” and the mobile price bar (₹299) are hard-coded; default quantity 2; stock note always “Only n left”	M
KL-04	Data	`ProductVariant` is never read; size does not change price; price filter ignores variants	M
KL-05	Filters	“Featured” and “Exclude Out Of Stock” match nothing	M
KL-06	Filters	Star-rating filter is not connected	M
KL-07	Filters	Tag matching is exact and case-sensitive; inconsistent tag casing and spelling (`diabitic-care`, `best seller` vs `best-seller`)	M
KL-08	Cart	Adding the same product with different size or message merges into one line	M
KL-09	Cart	Cart page and side sheet use different delivery and tax rules	M
KL-10	Cart	`originalPrice` is invented as price + ₹50	M
KL-11	Order	Cart items lack `creatorSlug`, so order items get no baker and no tasks are created	H
KL-12	Order	Server trusts client prices; no input validation; raw error text returned	H
KL-13	Order	Random six-digit order numbers can collide; no retry	M
KL-14	Order	No stock check or decrement	H
KL-15	Order	Success page `/checkout/success/{id}` does not exist	M
KL-16	Auth	No accounts; customer name and email are hard-coded	H
KL-17	Catalogue	Out-of-stock products can still be added to the cart	M
KL-18	Database	Live schema differs from `schema.prisma` and migrations (§4.4)	H
KL-19	Data	Stray tabs in a slug and a name, mixed-case slug, shared image IDs, name differs from slug	M
KL-20	Bakers	Baker list on the sidebar is hard-coded; `/bakers` ignores `minRating` and `location`; baker cards not clickable	L
KL-21	Home	Hero slides and baker names are static constants	L
KL-22	Errors	No error boundaries; unknown slug shows plain text “Not found”	M

ID	Area	Limitation	Sev
KL-23	Repository	Build output (`apps/**/.next`, 574 tracked files) and `.nx/` cache are committed; README is the default template with a stray UTF-16 line; stray files `pa`, `Product.json`, `useUpdateFilters_odd.ts`; debug `console.log` calls	M
KL-24	Quality	No CI; no tests for filters, store, pages or server action	M
KL-25	Accessibility	Generic alt text; unlabeled icon buttons; contrast not audited	M
KL-26	Search	Multi-word search fails (spaces turned into hyphens); each keystroke triggers a navigation; input not prefilled from URL	M
KL-27	Security	No rate limiting, bot protection or custom security headers	M
KL-28	Header	Logo is not a link; no mobile navigation observed in code **[CONFIRM]** on a phone	L
KL-29	Caching	`unstable_cache` usage should be reviewed against Next.js 16 guidance **[CONFIRM]**	L

14. Future Scope (Not Planned for Build)

Kept for completeness, in case the project is resumed.

Area	Possible work
Checkout and payments	Customer details, delivery address, payment gateway, confirmation page, email or SMS notifications
Accounts	Sign-in, order history, saved addresses
Catalogue	Use `ProductVariant` for size and price; sorting; managed tag list; fix filters (KL-05 to KL-07, KL-26)
Integrity	Server-side price validation, input validation, stock reservation, order number sequence (KL-11 to KL-14)
Bakers	Baker profile pages (`/bakers/[slug]`); baker operations app using `Task`; use `OutboxEvent` for event delivery
Content	Managed home sections and hero banners; rewrite About Us; reviews
Header	Call button for phone orders; font-size controls; local-language option
Quality	CI, end-to-end tests, error pages, SEO metadata, accessibility audit, monitoring
Hygiene	Remove committed build output; fix README; align live schema with migrations

Appendix A. Draft HLD/LLD PDF: Reconciliation

The earlier draft PDF (“BuzzFest Enterprise Architecture”) is a target-state design. It does not describe the built system.

PDF statement	As built
Redis cluster for caching	None; Next.js `unstable_cache` (60 s)
Cloudflare and Vercel Edge	Vercel hosting only
REST `POST /api/orders`, `PATCH /api/admin/orders/[id]`	No API routes; a Server Action creates orders; no admin API
Razorpay or Stripe checkout with webhook setting `PAID`	No payment integration
Server re-fetches prices	Prices come from the browser
`User` and `Address` models	Do not exist
`Product.title`, `basePrice`, `isAvailable`	`name`, `price`, `stock`

PDF statement	As built
`OrderItem.snapshotTitle`, `snapshotPrice`	`productName`, `priceAtPurchase`
Order statuses `CONFIRMED`, `BAKING`	Not in schema or database
Baker, Task, ProductVariant, OutboxEvent	Missing from the PDF
Kitchen dashboard	Not started

Appendix B. Glossary

Term	Meaning
App Router	Next.js routing system based on the `app/` folder
Server Component	React component rendered on the server; no client JavaScript
Server Action	A server function the browser can call directly (`"use server"`)
ISR / data cache	Time-based re-use of server results (`revalidate`)
Slug	URL-safe unique name, such as `priya-sharma`
Public ID	Cloudinary's identifier for an image
Neon	Serverless PostgreSQL provider
Prisma	TypeScript ORM and migration tool
Zustand	Small client-side state library
shadcn/ui	Copy-in component kit built on Radix UI and Tailwind
KL-nn	Known limitation reference (§13)

Appendix C. Screenshots to Add

Screenshots show the live behaviour that this document refers to. Upload these (desktop about 1440 px wide, phone about 390 px) and they can be inserted at the sections shown.

Figure	Capture	Section
A-1	Home, whole page, desktop	3.3
A-2	Products page with sidebar and one filter applied (URL visible)	5.1
A-3	Products page, empty result	5.1
A-4	Product detail, desktop	5.6
A-5	Cart with items and summary	5.4
A-6	Cart side sheet open	5.4
A-7	Browser developer tools showing the `buzz-fest-cart-storage` entry	5.3
A-8	Storybook component list	7.5
A-9	Result of pressing **Proceed to Pay** on the live site	5.5

Appendix D. Open Items for the Owner

1. What happens when **Proceed to Pay** is pressed on the live site (§5.5)? The code creates an order and redirects to a missing page.
2. Is the live Neon schema the intended one (§4.4)?
3. Confirm the Cloudinary variable name, Node version, and whether Vercel Analytics is enabled.
4. Were the `apps/` and `.nx/` folders meant to be committed (KL-23)?